

5 El procesador segmentado

La segmentación (*pipelining*) es una técnica por medio de la cual la arquitectura de un computador se divide en múltiples etapas, cada una de las cuales, se corresponde con una de las fases experimentadas por una instrucción desde que es leída de memoria hasta que sus resultados son escritos en el banco de registros. Este enfoque permite que distintas instrucciones ocupen distintas etapas de manera concurrente, explotando así el paralelismo a nivel de instrucción, y requiere introducir en la arquitectura registros de etapa con los que almacenar de manera temporal el contexto de una instrucción al final de cada etapa, es decir, el conjunto de datos y señales de control asociados a la instrucción, los cuales serán transferidos a la etapa inmediatamente posterior en el siguiente ciclo de reloj.

Puede verse en la Figura 5.1 un diagrama en el que se compara el flujo de ejecución de un computador monociclo frente al de uno segmentado.

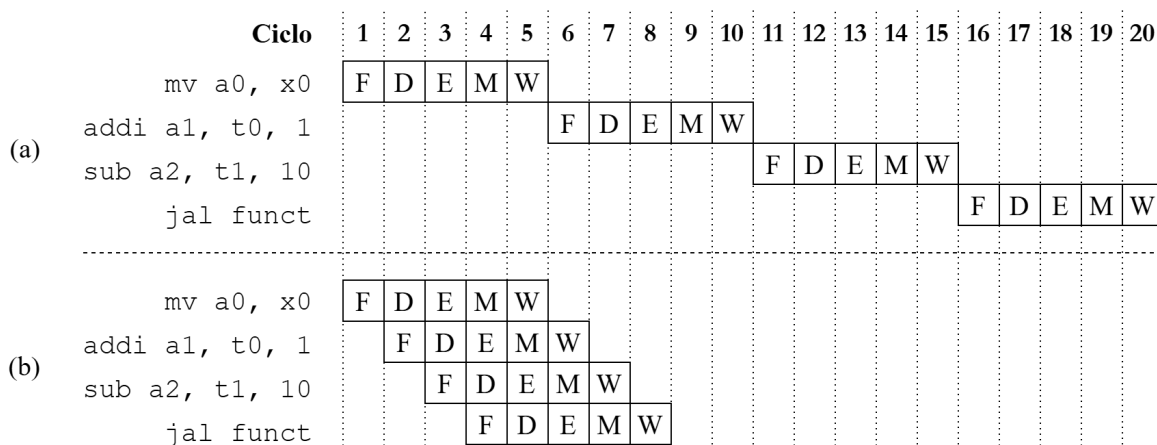


Figura 5.1: Ejecución de un programa en un computador con diseño monociclo (a) y segmentado (b)

Esta técnica conlleva una mayor complejidad arquitectónica y el surgimiento de una serie de riesgos o *hazards* generados bien por las dependencias de datos entre las instrucciones de un programa (ver Figuras 5.2 y 5.3), o bien por la ejecución de instrucciones de control que pueden o no alterar el flujo de ejecución de este (ver Figura 5.4). En función de las decisiones de diseño que se tomen para abordar la problemática que representan estos fenómenos, su aparición desembocará en la introducción de detenciones (*stalls*) en el flujo de ejecución, o la necesidad de incluir en el diseño mecanismos adicionales como el *forwarding*, consistente en el conexionado de determinadas señales de datos o de control hacia etapas anteriores en el pipeline.

En la Figura 5.2 aparecen identificados los distintos tipos de dependencias de datos que pueden darse durante la ejecución de un programa. La dependencia RAW (a) se da cuando uno de los operandos de una instrucción es un registro (`x1`, señalado en color morado) cuyo valor deberá ser previamente modificado

<code>add x1, x2, x3</code> <code>sub x4, x1, x5</code> (a)	<code>add x1, x2, x3</code> <code>add x1, x4, x5</code> (b)	<code>add x4, x1, x2</code> <code>add x1, x3, x5</code> (c)
---	---	---

Figura 5.2: Ejemplos de riesgos de datos. (a) RAW (*Read After Write*): lectura tras escritura. (b) WAW (*Write After Write*): escritura tras escritura. (c) WAR (*Write After Read*): escritura tras lectura

por otra instrucción separada de la primera **3 ciclos o menos**. Esto es así ya que la primera instrucción, la que lee el dato, deberá poder observar ese **dato** actualizado de modo que pueda elaborar un resultado coherente con el orden del programa. Es decir que, en un principio, la segunda instrucción no debería poder leer sus operandos (en su *decode*) hasta la escritura (*writeback*) del resultado calculado por la primera. Se ejemplifica esta situación por medio del diagrama presentado en la Figura 5.3.

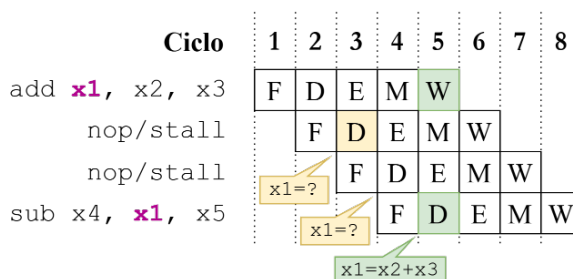


Figura 5.3: Flujo de ejecución de dos instrucciones implicadas en una dependencia RAW. Se señala en color verde el instante a partir del cual la segunda instrucción podrá leer el dato actualizado en el registro x1 (ciclo 5).

Los otros dos tipos de dependencias presentados, señalados con las letras *b* y *c*, no representan un riesgo real ya que dos instrucciones pueden leer o escribir de manera consecutiva sobre el mismo registro sin que ello suponga la pérdida de coherencia en el dato, al igual que sucede con las escrituras sobre un registro que fue previamente leído.

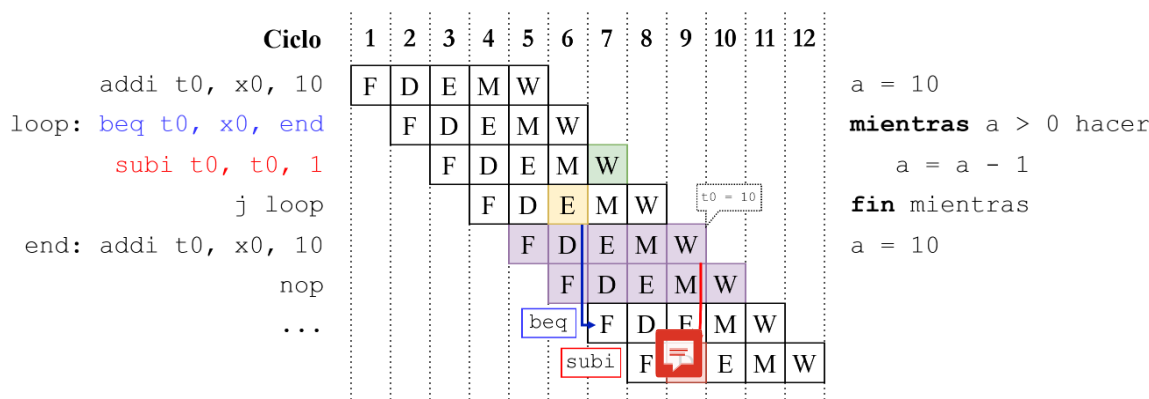


Figura 5.4: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada sin mecanismos para resolver riesgos de control.

Sobre el ejemplo de riesgo de control presentado en la Figura 5.4, el resultado de la instrucción de resta (`subi`) se escribe (writeback señalado en color verde) en el banco de registros antes incluso de evaluar la condición en la instrucción de salto, dejando el valor del registro `t0` en un estado inconsistente. Además, dado que el destino del salto de la instrucción `jal` no se calculará hasta su ejecución, habrá dos ciclos en los que se introducirán las dos instrucciones inmediatamente posteriores (señaladas en color morado), una de las cuales reinicia el valor de la variable `a` haciendo que el bucle no tenga final. El resultado de la instrucción `subi` en la primera iteración se sobrescribe con el del segundo `addi`.

Por último, y a pesar de las limitaciones comentadas, se debe destacar que la segmentación de instrucciones permite obtener un speedup teórico próximo al número de etapas en que se haya decidido segmentar la arquitectura, teniendo siempre en cuenta que aspectos como la presencia y resolución de los riesgos mencionados, la latencia en los accesos a memoria o el tiempo desigual entre los caminos críticos de las diferentes etapas, impedirán alcanzar ese rendimiento máximo ideal.

5.1. Arquitectura de un registro de etapa

La función de un registro de etapa es almacenar de manera transitoria el conjunto de señales de control y datos emitidos al término de una etapa de modo que, en el siguiente ciclo de reloj, esas mismas señales puedan ser transferidas a la etapa inmediatamente posterior en el pipeline. De este modo se consigue desacoplar temporalmente la ejecución de las distintas etapas permitiendo que varias instrucciones ocupen el pipeline de manera concurrente.

Desde el punto de vista de la implementación en Chisel, un registro de etapa como el mostrado en la Figura 5.5 se compone, en primer lugar, de un bundle en el que se definen las entradas y salidas de este. Estas se corresponderán con el conjunto de señales de control y datos que deban avanzar de una etapa a la siguiente. Posteriormente, cada entrada de datos/control se conectará a la entrada de un registro específico de esa señal y, la salida del registro, a su correspondiente salida en el bundle.

```

1 class FD extends Module {
2
3   val io = IO(new Bundle {
4     val f_instruction      = Input(UInt(Parameters.InstructionWidth))
5     val f_instruction_address = Input(UInt(Parameters.AddrWidth))
6
7     val d_instruction      = Output(UInt(Parameters.InstructionWidth))
8     val d_instruction_address = Output(UInt(Parameters.AddrWidth))
9   })
10
11   val instruction      = RegInit(0.U(Parameters.InstructionWidth))
12   val instruction_address = RegInit(0.U(Parameters.AddrWidth))
13
14   instruction      := io.f_instruction
15   instruction_address := io.f_instruction_address
16
17   io.d_instruction      := instruction
18   io.d_instruction_address := instruction_address
19 }

```

Figura 5.5: Implementación inicial del registro de etapa que separa las fases de *fetch* y decodificación.

5.2. Segmentando el núcleo en dos etapas

El primer paso hacia la segmentación del núcleo en las cinco etapas marcadas como objetivo del trabajo consistió en segmentarlo primeramente en dos, aislando la fase de *fetch* respecto de las unidades funcionales implicadas en el resto de etapas.

Esta división no conlleva la aparición de riesgos de datos puesto que tanto la escritura como la lectura de estos forman parte de la misma fase dentro del ciclo de procesamiento de las instrucciones. Sin embargo, esta segmentación sí da pie a que, durante la ejecución de un programa, puedan aparecer riesgos de control tal y como se muestra en el ejemplo presentado en la Figura 5.6, donde la ausencia de mecanismos que logren mitigar el riesgo de control provoca la lectura de la instrucción señalada en color rojo antes de que se haya evaluado siquiera la condición en la instrucción de salto, ejecutando así la instrucción de resta y depositando en el registro `x1` un valor incoherente con la lógica del programa.

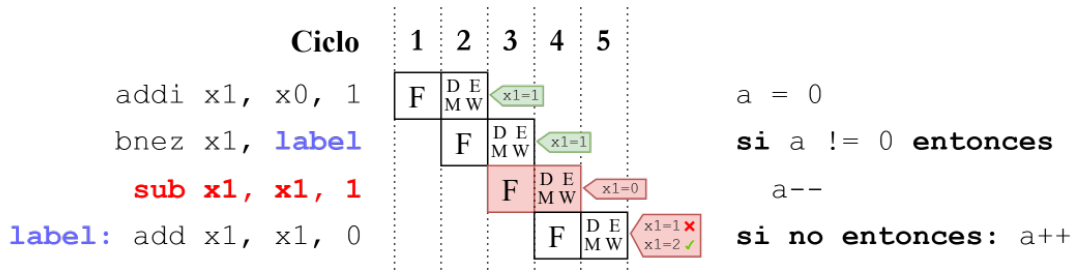


Figura 5.6: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada en dos etapas sin mecanismos para resolver riesgos de control.

Para verificar la existencia o no del riesgo de control tras la introducción en la arquitectura del primer registro de segmentación, se desarrolló un sencillo programa en lenguaje ensamblador el cual incorpora instrucciones de salto condicional, salto incondicional y una llamada a función. Este programa (cuyo código se muestra en la Figura 5.7) fue compilado, cargado en memoria y ejecutado en el computador simulado por medio del test de Chisel mostrado en la Figura 5.8. El test consta, en primer lugar, de un bucle de inicialización en el que se espera a que el cargador lea el contenido del binario y lo deposite en la memoria del computador. Posteriormente, se ejecuta un segundo bucle en el que se hace avanzar el contador 100 ciclos, a priori, suficientes como para recoger la ejecución completa del programa en la traza VCD resultante de la ejecución del test.

El primero de los problemas que se observó al examinar la traza generada por el test fue la falta de sincronismo entre el valor del contador de programa observado en una etapa determinada, y la instrucción observada en esa misma etapa, tal y como aparece señalado en color rojo en la Figura 5.9. En esa sección de la traza se muestran los cambios en el valor de las siguientes señales:

- `instr_addr@fetch`: la dirección de la instrucción presente en la fase de fetch.
- `instr_addr@DEMW`: la dirección de la instrucción en la fase siguiente al fetch.
- `instr@fetch`: la instrucción leída en la última fase de fetch.
- `instr@DEMW`: la instrucción leída en el anterior fetch.
- `fetch_io.jump_flag`: la señal que indica si la evaluación de una condición es positiva y debe leerse por tanto en el siguiente ciclo, no la instrucción siguiente al contador de programa actual, si no el *target* del salto. Tanto el *target* como esta señal se originan en la fase siguiente al fetch, como parte de la fase de ejecución (módulo *Execute*).
- `ex_io.immediate`: el offset empleado en el cálculo de la dirección efectiva de salto, también como parte de la fase de ejecución.
- `ex_io.instr_addr`: la dirección base del salto.
- `fetch_io.jump_addr`: la dirección efectiva del salto. Esta señal llega al módulo *InstructionFetch* desde *Execute*.

Código máquina	Ensamblador	Pseudocódigo
<pre> 00000000 <_start>: 0: 00000313 4: 00400393 00000008 <loop_start>: 8: 0063ca63 c: 00030513 10: 010000ef 14: 00050313 18: ff1ff06f 0000001c <loop_end>: 1c: 0000006f 00000020 <suma_uno>: 20: 00050293 24: 00128293 28: 00028513 2c: 00008067 </pre>	<pre> li t1,0 li t2,4 blt t2,t1,loop_end mv a0,t1 jal suma_uno mv t1,a0 j loop_start j loop_end mv t0,a0 addi t0,t0,1 mv a0,t0 ret </pre>	<pre> t1 = 0 t2 = 4 mientras t1 < t2 t1 = suma_uno(t1) función suma_uno(x) retornar x + 1 </pre>

Figura 5.7: Desensamblado, código ensamblador y pseudocódigo del programa empleado en la depuración de la arquitectura.

```

1 class SegRegFDTest extends AnyFlatSpec with ChiselScalatestTester {
2   behavior.of("Pipelined CPU")
3   it should "run a simple program" in {
4     test(new TestTopModule("simple_subroutine.asmbin"))
5       .withAnnotations(TestAnnotations.annos) { c =>
6         while(!c.io.instruction_valid.peek().litToBoolean) {
7           c.clock.step()
8           c.io.mem_debug_read_address.poke(4.U)
9         }
10
11         for (i <- 1 to 100) {
12           c.clock.step()
13           c.io.mem_debug_read_address.poke((i * 4).U)
14         }
15
16         c.io.regs_debug_read_address.poke(6.U) // x6 => t1
17         c.io.regs_debug_read_data.expect(5.U)
18       }
19   }
20 }

```

Figura 5.8: Código del test con el que se pone a prueba el funcionamiento de un programa sencillo sobre el núcleo segmentado en dos fases.

La consecuencia que esta falta de sincronismo tiene sobre el flujo de ejecución de los programas es que instrucciones de salto no pueden realizar correctamente el cómputo de la dirección efectiva de este, al contar en la fase de ejecución con el contador de programa de la instrucción siguiente al salto, y no la del propio salto. Tal y como se observa en la traza de la Figura 5.9, al comienzo del ciclo marcado en la posición en que se encuentra el indicador vertical (43 ps), la instrucción `jal suma_uno (0x010000EF @`

0x1010¹, ver señal `instr@DEMW`), observa los siguientes operandos en el cálculo del destino del salto:

- `ex_io_immediate`: 0x10
- `instr_addr@DEMW`: 0x1014

Puede observarse que la dirección base del salto no es la correspondiente a la instrucción `jal`, si no la de la instrucción `mv` que la sucede en el programa, provocando que el salto no sea al comienzo de la subrutina `suma_uno`, si no a la segunda instrucción dentro de esta.

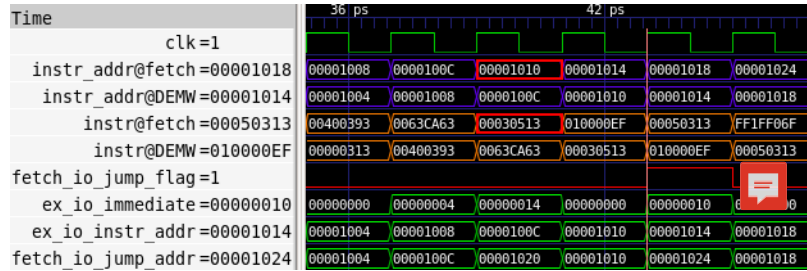


Figura 5.9: Sección de la traza obtenida tras la ejecución del test mostrado en la Figura 5.7

El diseño original emplea como memoria una instancia de la clase *SyncReadMem*. Este tipo de memoria no permite realizar lecturas de manera asíncrona, lo cual implica que, cuando se solicita una lectura, el dato leído no sale de la memoria hasta el siguiente flanco ascendente de reloj, impidiendo que se mantenga la sincronía entre las instrucciones en cada fase y la dirección de la instrucción observada en esa misma fase. En el computador objeto de este trabajo, este comportamiento de la memoria se materializa en que, cuando se envía la dirección de una instrucción para leerla, esa lectura no será efectiva hasta el ciclo siguiente al envío de la dirección. De ahí que, por ejemplo, la dirección de una instrucción en la fase de fetch (señal `instr_addr@fetch`) no se corresponda con la instrucción observada en esa misma fase (señal `instr@fetch`), si no con la instrucción inmediatamente anterior a la solicitada.

Para solucionar esta problemática se cambia, dentro de la clase *Memory*, el tipo de memoria a una de la clase *Mem*, de forma que la lectura del dato se lleve a cabo en el instante mismo en que se solicita. De este modo, tal y como puede observarse en la traza presentada en la Figura 5.10, el valor de la dirección de la instrucción en cada fase se corresponde con la de la instrucción que se está procesando en esa misma fase, de tal modo que ahora el cálculo del *target* en la instrucción de salto sea correcto ($0x1010$ (`ex_io_instr_addr`) + $0x10$ (`ex_io_immediate`) = $0x1020$ (`fetch_io_jump_addr`) → `suma_uno`).

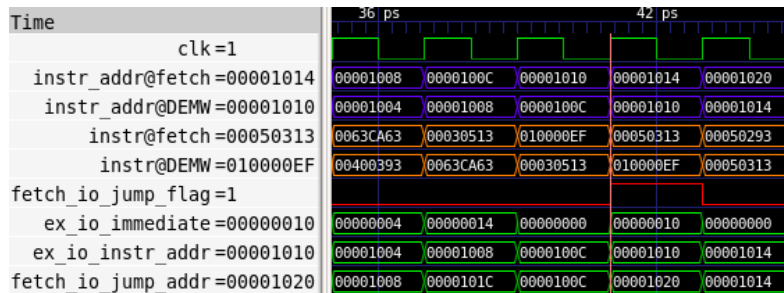


Figura 5.10: Misma sección de la traza pero con una instancia de la clase *Mem* como entidad de memoria en lugar de *SyncReadMem*.

¹El punto de entrada de los programas se encuentra en la dirección 0x1000. La dirección de la instrucción es el valor del punto de entrada sumado al offset mostrado en la primera columna de la Figura 5.7

A pesar de esta corrección, puede observarse que el flujo de ejecución del programa sigue siendo erróneo ya que, siguiendo a la instrucción de salto (tras la marca vertical), en la segunda fase logra introducirse y ejecutarse la instrucción inmediatamente posterior, la cual es un `mv t1, a0` (0x00050313 @ 0x1014). En el caso concreto de la primera iteración del bucle, la ejecución de esta instrucción es inócua ya que en ese punto los registros `t1` y `a0` albergan el mismo valor. No obstante a esto, la inserción de la instrucción siguiente al salto toma una relevancia significativa cuando se trata de otra instrucción de salto, tal y como ocurre tras el salto incondicional al final del bucle. La ejecución de este segundo salto, el cual forma parte del bucle de final de programa, provoca que este llegue a término tras la primera iteración (véase Figura 5.9).

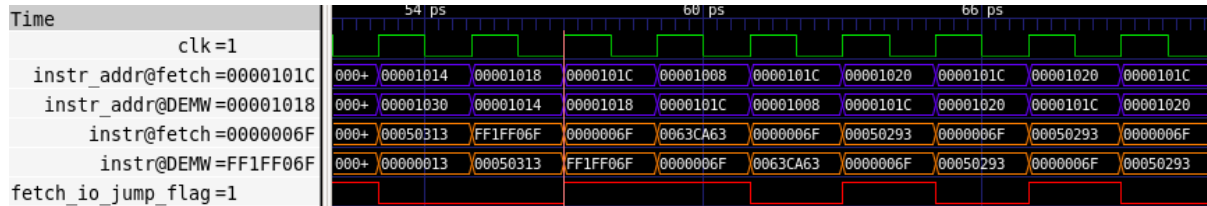


Figura 5.11: Sección de la traza en la que se muestra la pronta ejecución del bucle final.

La solución final para resolver esta problemática consiste en la introducción de un par de modificaciones en la fase de fetch para hacer que la arquitectura introduzca una operación `nop` tras cada instrucción de salto en cuya segunda fase se determine que este deba ser tomado. Estas modificaciones se realizan sobre la lógica que hace avanzar el contador de programa.

En la nueva implementación, tal y como se observa en el `diff` presentado en la Figura 5.12, se introduce un `nop` en el pipeline siempre y cuando se cumplan las dos condiciones siguientes: en primer lugar, que el cargador no haya terminado su trabajo y, en segundo lugar, que en la segunda fase de una instrucción de salto se haya determinado que este deba ser tomado (originalmente solo se contemplaba la primera condición).

```

val pc = RegInit(ProgramCounter.EntryAddress)

- when(io.instruction_valid) {
-   pc          := Mux(io.jump_flag_id, io.jump_address_id, pc + 4.U)
-   io.instruction := io.instruction_read_data
+ when(io.instruction_valid && !io.jump_flag_id) {
+   pc          := pc + 4.U
+   io.instruction := io.instruction_read_data
+ } .otherwise {
-   pc          := pc
+   pc          := io.jump_address_id
+   io.instruction := 0x00000013.U
+ }

io.instruction_address := pc

```

Figura 5.12: Comparación entre la implementación original del módulo `InstructionFetch` y su adaptación para resolver riesgos de control.

5.3. Segmentando el resto del núcleo

En un primer momento se optó por aplicar un enfoque incremental en lo que respecta a la inserción de los registros de etapa. No obstante, este enfoque se aplicó tan solo en la integración del primer registro de etapa para separar la fase de *fetch* del resto del pipeline. Esto fue así puesto que, de haber seguido integrando registros de etapa uno a uno, verificando el funcionamiento del núcleo con dos, tres y cuatro registros, el proceso de desarrollo se hubiera dilatado en el tiempo más allá de lo deseado inicialmente ya que, para cada una de las integraciones de un registro de segmentación a partir de aquel que separa las fases de *fetch* y decodificación, hubiera sido preciso diseñar una solución distinta al problema que generan las dependencias de datos y de control entre las instrucciones que atraviesan el pipeline.

Es por ello por lo que se consideró que resultaría más eficiente integrar los registros de segmentación siguientes al de *fetch*-decodificación a la vez, desarrollando una única solución al problema generado por las dependencias de datos y de control.

Esta solución se ha materializado en el código de la clase *HazardUnit*, la cual modela la lógica que gobierna el comportamiento de una unidad de amenazas. El cometido de esta pieza es observar el estado de la arquitectura a cada ciclo para identificar y resolver las dependencias de datos o de control que pudieran originarse durante la ejecución de los programas. A continuación se detalla el método empleado en la resolución de los riesgos de datos del tipo RAW y de control, así como las señales de la unidad de amenazas involucradas en el proceso de detección y mitigación del riesgo.

5.3.1. Resolución de riesgos de datos (RAW)

En el caso de una arquitectura segmentada en cinco fases, un riesgo de datos del tipo RAW se da cuando en un programa existe una pareja de instrucciones consecutivas para las cuales se cumple que, la segunda de ellas, posee como registro origen en alguno de sus operandos el registro de destino de la primera, pudiendo encontrarse separadas una distancia de 1 o 2 ciclos. Esta casuística queda ilustrada por medio del ejemplo presentado en la Figura 5.13, donde la instrucción `subi` debe esperar al *writeback* del `addi` para poder leer del registro `x10` un valor actualizado y consistente con la lógica del programa.

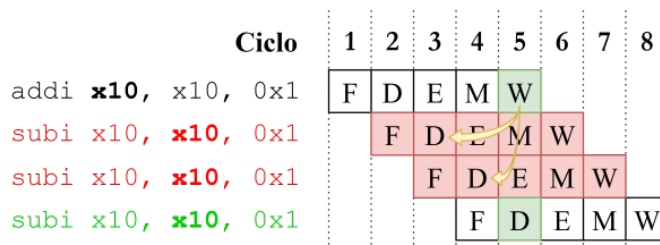


Figura 5.13: Ejemplo de una dependencia RAW en una arquitectura segmentada en cinco fases.

La solución (que no sea introducir *nops*) al problema que supone la ocurrencia de esta clase de escenarios pasa por la aplicación de una técnica denominada *forwarding*. Esta técnica consiste en adelantar el resultado de una instrucción, ya sea aritmético-lógica o una lectura de memoria, directamente a la fase de ejecución de las instrucciones que deban emplear ese dato como operando. Esto queda ejemplificado, también, en el diagrama de la Figura 5.13 por medio de las flechas de color amarillo: la primera instrucción `subi` podría ejecutar sin problemas y sin perder ni un solo ciclo si, desde el *writeback* del `addi` se le adelantara a su fase de ejecución el resultado a escribir en el registro `x10`, y de igual modo ocurriría con el segundo `subi`.

A continuación, se indicarán las soluciones propuestas al problema de las dependencias de datos del tipo RAW, tanto en el caso de las instrucciones aritmético-lógicas como en el caso de la instrucción de lectura de memoria:

Riesgos causados por la sucesión de instrucciones aritmético-lógicas

Las señales de la **unidad de amenazas** implicadas en la detección de esta clase de riesgos son:

- **rs1_ex**: la dirección del registro empleado como primer operando de la instrucción, observado en la fase de ejecución.
- **rs2_ex**: la dirección del registro empleado como segundo operando de la instrucción, observado en la fase de ejecución.
- **rd_mem**: la dirección del registro destino, observado en la fase de acceso a memoria.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.

La unidad de amenazas gobierna, además, las señales de selección de dos multiplexores emplazados, cada uno, en una de las entradas de la ALU:

- **alu_rs1_src**: controla la selección del primer operando de la ALU, que puede ser un dato leído a partir del banco de registros por parte de la instrucción en fase de ejecución, o un dato adelantado desde la fase de memoria o de *writeback*.
- **alu_rs2_src**: exactamente igual que la anterior, pero aplicada al segundo operando de la ALU.

La lógica que rige el proceso de detección de las dependencias por parte de la unidad de amenazas es tal y como se indica en la Figura 5.14, siendo el código aplicable tanto en el caso en que la dependencia se diese con el primer registro operando, como si esta se diese con el segundo.

Más allá de lo auto-explicativo del pseudo-código, cabe reseñar el por qué de la condición en la que se evalúa si el registro origen es x0 y es que, de darse una dependencia RAW en la que este registro se encontrase involucrado, no tendría sentido realizar un *forwarding* de un registro cuyo valor es constantemente 0, por lo que se evita solventar esta falsa **dependencia**.

Código	Pseudo-código
<pre> io.alu_rs1_src := Mux(io.rs1_ex == io.rd_mem) & io.reg_wr_enable_mem) & (io.rs1_ex != 0.U), RsFwSel.ALUResultMemFw, Mux(((io.rs1_ex == io.rd_wb) & io.reg_wr_enable_wb) & (io.rs1_ex != 0.U), RsFwSel.ALUResultWbFw, RsFwSel.RegFileData)) </pre>	<pre> # Detección dependencia RAW entre Ex y Mem si primer reg. origen @ Ex = reg. destino @ Mem y escritura regfile @ Mem habilitada y primer reg. origen @ Ex != x0 entonces 1er operando ALU = resultado adelantado desde Mem # Detección dependencia RAW entre Ex y Wb si no si primer reg. origen @ Ex = reg. destino @ Wb y escritura regfile @ Wb habilitada y primer reg. origen @ Ex != x0 entonces 1er operando ALU = resultado adelantado desde Wb si no entonces 1er operando ALU = dato leído por la instrucción </pre>

Figura 5.14: Código y pseudo-código de la lógica empleada en la detección y mitigación de riesgos de datos del tipo RAW en los que se encuentra involucrado el primer registro origen de una instrucción.

Dependencias en las que se encuentra involucrada la instrucción 'lw'

La presencia de la instrucción `lw` en una dependencia de datos del tipo RAW genera un caso especial que no puede resolverse por medio de la metodología seguida en el apartado anterior. Esto es debido a que el resultado de la instrucción `lw` no se genera hasta haber completado la lectura de un dato de memoria, lo cual se produce al término de la fase de memoria de la instrucción. En ese momento la instrucción dependiente inmediatamente posterior al `lw` ya habrá completado su fase de ejecución, pero lo habrá hecho con un operando erróneo.

La técnica con la que se resuelve la dependencia consiste en detener el procesamiento de las instrucciones en fases de *fetch* y decodificación cada vez que esta sea detectada. Esto se consigue reteniendo los datos de los registros de segmentación que separan las fases de *fetch*-decodificación, y decodificación-ejecución. Esta retención dura tan solo un ciclo, de tal modo que, cuando la instrucción dependiente que no es el `lw` pase a la fase de ejecución, la unidad de amenazas tratará la dependencia como si de cualquier otra dependencia de datos se tratase, adelantando desde el writeback del `lw` el valor leído de la memoria, a la entrada de la ALU que corresponda. Se proporciona en la Figura 5.15 una representación de la resolución del riesgo de datos por medio de esta técnica.

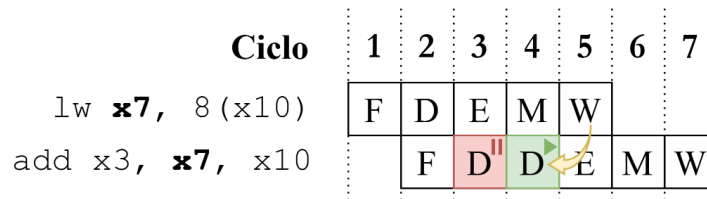


Figura 5.15: Flujo de ejecución de dos instrucciones implicadas en una dependencia de datos de tipo RAW, una de las cuales es la instrucción `lw`.

La solución implementada (Figura 5.16) ha consistido en hacer que la unidad de amenazas identifique, en primer lugar, si una instrucción en fase de ejecución va a provocar la escritura en el banco de registros de un dato leído de memoria. Esto se consigue observando el bit menos significativo de la señal `ex_regs_write_src`, la cual controla la salida del multiplexor en la fase de *writeback* con el que se ecoge el dato a escribir en el banco de registros. La salida de este multiplexor podrá ser un resultado calculado en la ALU, la dirección de retorno de un salto o el dato leído en una posición de memoria, siendo en este último caso, y por implementación, el valor del bit menos significativo de la señal igual a 1. Esta es condición suficiente y necesaria para asegurar que la instrucción en fase de ejecución es un `lw`. A continuación, en la segunda condición de la sentencia 'y', se realiza la detección de la dependencia de datos RAW con la instrucción inmediatamente posterior, de igual modo que se hacía en los casos comentados en el apartado anterior.

El resultado de estas comprobaciones es la señal `lw_stall`, que gobierna la parada del contador de programa y los registros de segmentación indicados anteriormente.

```

1 val lw_stall = io.ex_regs_write_src(0) &
2               ((io.rs1_d === io.rd_ex) | (io.rs2_d === io.rd_ex))
3
4 io.pc_stall    := lw_stall
5 io.srFD_stall  := lw_stall
6 io.srDE_flush  := lw_stall

```

Figura 5.16: Cómputo de la señal `lw_stall`.

5.3.2. Resolución de riesgos de control

Una solución sencilla y eficaz al problema generado por los riesgos de control es limpiar el contenido de los registros de segmentación entre las etapas de fetch-decodificación y decodificación-ejecución cada vez que, en la fase de ejecución de una instrucción de salto, se determina que este deba ser tomado. De este modo, tal y como se observa en el ejemplo de la Figura 5.17, todas las instrucciones (máximo 2) que logren introducirse en el pipeline tras la instrucción de salto, no se procesarán completamente y su paso por la arquitectura habrá sido totalmente inocuo.

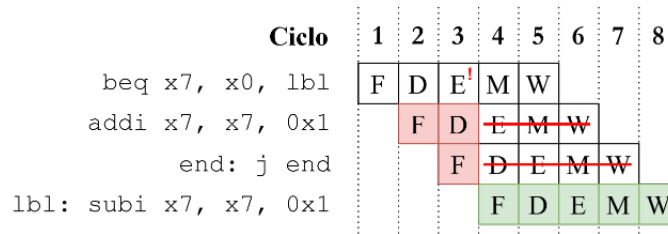


Figura 5.17: Diagrama ilustrativo del proceso de mitigación de los riesgos de control.

Esta solución se implementa por medio del fragmenteo de código de la unidad de amenazas mostrado en la Figura 5.18, donde se establece que el valor de la señal de borrado de los registros de segmentación indicados (señales `srDE_flush` y `srFD_flush`), será igual al valor del flag de salto observado en la fase de ejecución.

```

1 io.srDE_flush := lw_stall | io.ex_jump_flag
2 io.srFD_flush := io.ex_jump_flag

```

Figura 5.18: Configuración de la señal de borrado de los registros de segmentación que separan las fases de fetch-decodificación y decodificación-ejecución, conforme al valor del flag de salto en fase de ejecución.